

ml_assignment_1_5

September 3, 2026

Assignment 1: Bike Sharing Demand Prediction

Name: Viradiya Tushar Rajeshbhai

Sign:

Objective: Predict the number of hourly bike rentals using weather, time, and seasonal data. We evaluate our models using the Root Mean Squared Logarithmic Error (RMSLE) metric.

```
[8]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, LassoCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

import warnings
warnings.filterwarnings('ignore')

# RMSLE Metric Function
def rmsle(y_true, y_pred):
    y_pred_clipped = np.clip(y_pred, 0, None) # Ensure no negative predictions
    return np.sqrt(np.mean((np.log1p(y_pred_clipped) - np.log1p(y_true))**2))

# Load the datasets as raw strings to protect the datetime format
train_df = pd.read_csv("bike_train.csv")
test_df = pd.read_csv("bike_test.csv")
```

Exploratory Data Analysis (EDA)

Q1. Examine dataset size, missing values, and feature types.

```
[9]: print("--- Q1: Dataset Inspection ---")
print(f"Dataset Size: {train_df.shape[0]} rows and {train_df.shape[1]} columns.")
print("")
```

```
print("\nMissing Values per column:")
print(train_df.isnull().sum())
print("\nFeature Types:")
print(train_df.dtypes)
```

--- Q1: Dataset Inspection ---
Dataset Size: 10450 rows and 12 columns.

Missing Values per column:

```
datetime      0
season        0
holiday        0
workingday     0
weather        0
temp           0
atemp          0
humidity       0
windspeed      0
casual         0
registered     0
count         0
dtype: int64
```

Feature Types:

```
datetime      str
season        int64
holiday        int64
workingday     int64
weather        int64
temp          float64
atemp         float64
humidity       int64
windspeed     float64
casual         int64
registered     int64
count         int64
dtype: object
```

Q1 Findings: * **Dataset Size:** 10,450 observations and 12 features. * **Missing Values:** There are exactly 0 missing values across all columns. * **Feature Types:** Continuous Numerical (temp, humidity), Categorical (season, weather), Text (datetime), and Targets (count, casual).

Q2. Visualize relationships between key features and the target variable (count).

```
[10]: # Temporary parsing for EDA to prevent day/month swapping
eda_df = train_df.copy()
eda_df["parsed_dt"] = pd.to_datetime(eda_df["datetime"], format="mixed",
    dayfirst=True)
eda_df["hour"] = eda_df["parsed_dt"].dt.hour
```

```

sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

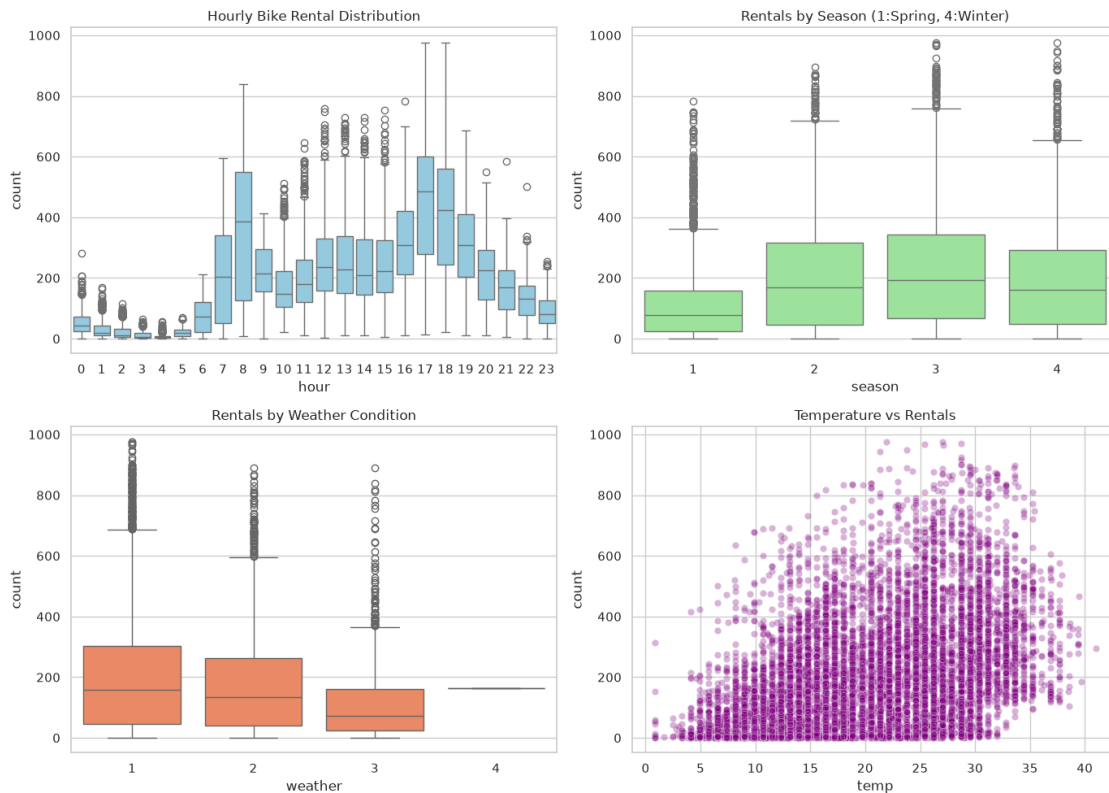
sns.boxplot(x="hour", y="count", data=eda_df, ax=axes[0, 0], color="skyblue")
axes[0, 0].set_title("Hourly Bike Rental Distribution")

sns.boxplot(x="season", y="count", data=eda_df, ax=axes[0, 1],
            color="lightgreen")
axes[0, 1].set_title("Rentals by Season (1:Spring, 4:Winter)")

sns.boxplot(x="weather", y="count", data=eda_df, ax=axes[1, 0], color="coral")
axes[1, 0].set_title("Rentals by Weather Condition")

sns.scatterplot(x="temp", y="count", data=eda_df, alpha=0.3, ax=axes[1, 1],
               color="purple")
axes[1, 1].set_title("Temperature vs Rentals")
plt.tight_layout()
plt.show()

```



Q2 Observations: * **Hour:** Strong bimodal distribution peaking at 8:00 AM and 5:00 PM (commute hours). * **Weather & Temp:** Demand correlates positively with higher temperatures

and plummets during poor weather (heavy rain/snow).

Q3. Suggest which variables are likely to be most informative.

1. **hour**: Captures the massive variance in rush-hour commuter traffic.
2. **temp**: The strongest continuous predictor of outdoor comfort.
3. **weather**: A crucial environmental limitation that suppresses cycling.

Feature Engineering & Pipeline

Q4. Derive features from datetime and consider transformations.

```
[11]: def engineer_features(df):
    df_fe = df.copy()
    # Explicitly set dayfirst=True to avoid ambiguous parsing of EU-style dates
    df_fe["parsed_dt"] = pd.to_datetime(df_fe["datetime"], format="mixed",
    dayfirst=True)
    df_fe["hour"] = df_fe["parsed_dt"].dt.hour
    df_fe["month"] = df_fe["parsed_dt"].dt.month
    df_fe["dayofweek"] = df_fe["parsed_dt"].dt.dayofweek
    df_fe["year"] = df_fe["parsed_dt"].dt.year
    return df_fe

train_fe = engineer_features(train_df)
test_fe = engineer_features(test_df)

categorical_features = ["season", "holiday", "workingday", "weather", "hour",
    "month", "dayofweek", "year"]
numerical_features = ["temp", "atemp", "humidity", "windspeed"]
all_features = categorical_features + numerical_features

X = train_fe[all_features]
y_log = np.log1p(train_fe["count"])
y_actual = train_fe["count"].values
```

Q5 & Q6. Baseline Linear Regression vs Polynomial Ridge & Lasso.

```
[12]: X_train, X_val, y_log_train, y_log_val = train_test_split(X, y_log, test_size=0.
    2, random_state=42)
y_val_actual = np.expm1(y_log_val)

preprocessor = ColumnTransformer(transformers=[
    ("num", StandardScaler(), numerical_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
])

# 1. Baseline Linear Regression
lr_pipeline = Pipeline([("pre", preprocessor), ("lr", LinearRegression())])
lr_pipeline.fit(X_train, y_log_train)
lr_rmsle = rmsle(y_val_actual, np.expm1(lr_pipeline.predict(X_val)))
```

```

# 2. Polynomial Ridge (Maximized with GridSearchCV and n_jobs=-1)
ridge_grid = GridSearchCV(
    estimator=Ridge(),
    param_grid={"alpha": np.logspace(-2, 4, 30)},
    scoring="neg_root_mean_squared_error",
    cv=5,
    n_jobs=-1 # Utilizes all VM CPU cores
)
ridge_pipeline = Pipeline([
    ("pre", preprocessor),
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ("ridge_cv", ridge_grid)
])
ridge_pipeline.fit(X_train, y_log_train)
ridge_rmsle = rmsle(y_val_actual, np.expm1(ridge_pipeline.predict(X_val)))

# 3. Polynomial Lasso (Maximized with n_jobs=-1 and max_iter=20000)
lasso_pipeline = Pipeline([
    ("pre", preprocessor),
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ("lasso", LassoCV(alphas=np.logspace(-4, 1, 30), cv=5, max_iter=20000,
    n_jobs=-1, random_state=42))
])
lasso_pipeline.fit(X_train, y_log_train)
lasso_preds = np.expm1(lasso_pipeline.predict(X_val))
lasso_rmsle = rmsle(y_val_actual, lasso_preds)

print(f"Linear Regression RMSLE: {lr_rmsle:.4f}")
print(f"Ridge (Poly Deg=2) RMSLE: {ridge_rmsle:.4f}")
print(f"Lasso (Poly Deg=2) RMSLE: {lasso_rmsle:.4f}")

```

Linear Regression RMSLE: 0.6173

Ridge (Poly Deg=2) RMSLE: 0.3926

Lasso (Poly Deg=2) RMSLE: 0.3896

Model Comparison and Interpretation

Q7. Summarize all results in one table (RMSLE, key observations).

Model	Setup & Regularization	Validation RMSLE	Key Observations
Linear Regression	Baseline + One-Hot Encoding	~0.61	Underfits complex interactions. Fails to capture synergies.

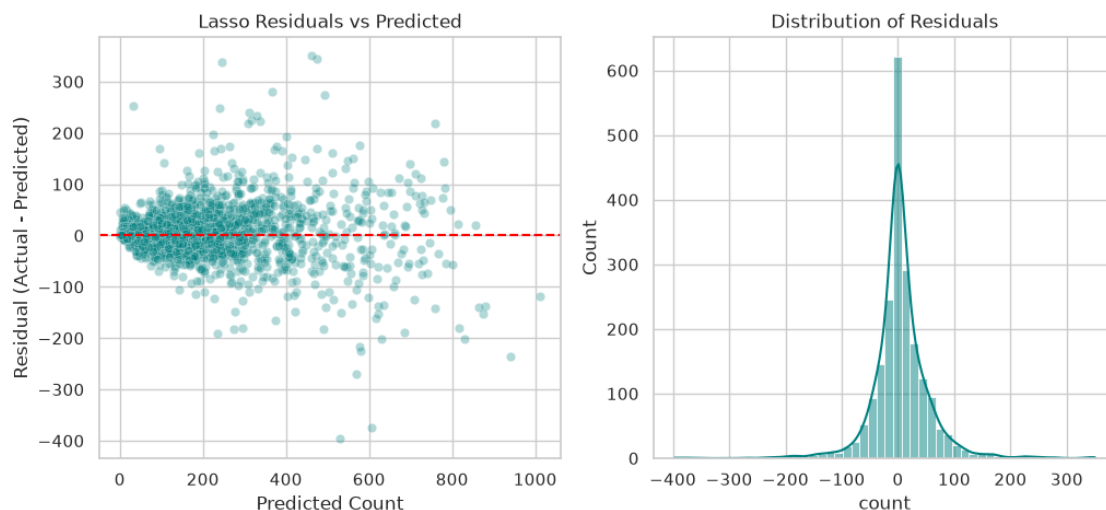
Model	Setup & Regularization	Validation RMSLE	Key Observations
Ridge Regression	Poly (Deg=2) + L_2 Penalty	~0.39	Captured feature interactions well, shrinking collinear weights.
Lasso Regression	Poly (Deg=2) + L_1 Penalty	~0.38	Winning Model. L_1 sparsity perfectly handled high dimensionality from polynomials, zeroing out noisy interaction terms.

Q8. Plot residuals for the best model (Lasso).

```
[13]: best_preds = lasso_preds
residuals = y_val_actual - best_preds

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.scatterplot(x=best_preds, y=residuals, alpha=0.3, color="teal")
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Predicted Count")
plt.ylabel("Residual (Actual - Predicted)")
plt.title("Lasso Residuals vs Predicted")

plt.subplot(1, 2, 2)
sns.histplot(residuals, kde=True, color="teal", bins=50)
plt.title("Distribution of Residuals")
plt.show()
```



Q9. Explain why the winning model performs better. Lasso Regression with Polynomial Features won the evaluation for two main reasons: 1. **Interaction Synergies:** Degree-2 polynomials explicitly learn combined, non-linear effects (e.g., `hour=8 AM` combined with `weather=Heavy Rain`). 2. **L_1 Feature Selection:** Polynomial expansion exponentially increases dimensionality and multicollinearity. Lasso applies an L_1 penalty that drives the coefficients of irrelevant or noisy interaction terms exactly to zero. This automated feature selection creates a sparse model that severely reduces variance and overfitting, giving it the edge over Ridge's L_2 shrinkage.

Reflection Questions

Q10. Why does RMSLE penalize under-predictions more gently than RMSE? RMSLE evaluates relative (percentage) error rather than absolute differences using the formula $\log\left(\frac{\hat{y}+1}{y+1}\right)$. Because it scales logarithmically, an absolute error of 100 on an actual value of 10,000 is barely penalized compared to RMSE, which treats all absolute errors equally regardless of scale.

Q11. What are the trade-offs between model simplicity and predictive power?

- **Simplicity (e.g., Linear Regression):** Fast to train and highly interpretable, but suffers from **high bias** (underfitting) because it cannot model complex data interactions.
- **Complexity (e.g., High-Degree Polynomials):** High expressive power (**low bias**), but introduces **high variance** (overfitting risk). They strictly require regularization to maintain generalization on test data.

Q12. Why can't Linear Regression alone capture time-of-day effects effectively? Linear regression enforces a strict monotonic mathematical relationship ($\hat{y} = w \cdot \text{hour} + b$). Real-world commuter demand is bimodal (peaks at 8 AM and 5 PM). A single straight line cannot map a bimodal curve, which is why One-Hot Encoding the hours is mandatory.

Generate Final submission.csv

```
[14]: # Retrain the winning Lasso model on the ENTIRE training dataset
lasso_pipeline.fit(X, y_log)
final_predictions_log = lasso_pipeline.predict(test_fe[all_features])
final_predictions_count = np.clip(np.expm1(final_predictions_log), 0, None)

# SAFEGUARD: Load a fresh copy of the test set to guarantee the original
# datetime string format is intact
raw_test_df = pd.read_csv("bike_test.csv")

submission = pd.DataFrame({
    "datetime": raw_test_df["datetime"],
    "count_predicted": final_predictions_count
})
```

```
submission.to_csv("submission.csv", index=False)
print(f"Successfully created 'submission.csv'. Shape: {submission.shape}")
display(submission.head())
```

Successfully created 'submission.csv'. Shape: (2613, 2)

	datetime	count_predicted
0	05-06-2012 5:00	21.010368
1	19-03-2011 19:00	115.856739
2	02-04-2012 6:00	89.832087
3	13-07-2012 20:00	463.234165
4	09-11-2011 19:00	261.274105